*Bioinformatics Advances*, YYYY, 0–0
doi: 10.1093/Bioinformatics Advances/xxxxx
Advance Access Publication Date: DD Month YYYY
Manuscript Category

*Original Article*

# metbit: An Integrated Python Framework for Reproducible NMR Metabolomics from Spectral Processing to Biological Interpretation

Theerayut Bubpamala[1,2,*] and Chotika Chatgasem[2]

[1] Medical Biochemistry and Molecular Biology Graduate Study Program, Faculty of Medicine, Khon Kaen University, Khon Kaen, Thailand

[2] kawa-technology, Independent Research & Development

[*] Corresponding author. kawa-technology, Independent Research & Development. E-mail: theerayut_aeiw_123@hotmail.com

Associate Editor: B. Theerayut

## Abstract

**Motivation:** Untargeted $^1$H NMR metabolomics requires reproducible integration of spectral preprocessing, normalisation, chemometrics, validation, and biological interpretation. These steps often remain split across vendor software and statistical tools, limiting reproducibility and scalability as cohort sizes approach biobank scale.

**Results:** We present metbit version 9.0.0, an open-source Python package that unifies raw FID NMR preprocessing, probabilistic quotient normalisation, multiplicative scatter correction, icoshift alignment, PCA, exploratory OPLS-DA, VIP scoring, statistical total correlation spectroscopy (STOCSY), and interactive visualisation in a scriptable workflow. Version 9.0.0 introduces a four-tier auto-dispatch compute backend that selects GPU acceleration through CuPy or PyTorch, a native C extension with optional OpenMP, multiprocessing, or memory-bounded chunked NumPy. Benchmarks on an Apple M5 Pro demonstrated up to 2,680-fold faster VIP computation and 47-fold faster ChunkedSTOCSY. Application to 132 MTBLS1 public urine spectra confirmed complete raw-to-OPLS-DA processing. A 305-test suite verifies functional correctness, statistical validity, and performance regression.

**Availability and implementation:** metbit is MIT-licensed and available at https://github.com/aeiwz/metbit (pip install metbit). Full documentation is at https://metbit-docs.vercel.app.

Keywords: NMR metabolomics; chemometrics; OPLS-DA; STOCSY; open-source bioinformatics; large-scale computation

## 1. Introduction

Proton nuclear magnetic resonance ($^1$H NMR) spectroscopy occupies a central role in untargeted metabolomics because of its quantitative accuracy, non-destructive sample handling, structural information content, and high analytical reproducibility (Moco, 2022; Gowda and Raftery, 2023; Powers et al., 2024). A single acquisition produces a spectrum containing thousands of chemical-shift variables that collectively encode resonances from endogenous metabolites. When spectra are collected across biological groups, chemometric analysis of the resulting data matrix can reveal metabolic signatures associated with phenotypes, exposures, and disease states (Huang et al., 2022; Gowda and Raftery, 2023). Recent reviews document the expanding use of NMR metabolomics in clinical phenotyping and disease research while emphasising the importance of standardised acquisition, processing, and statistical analysis (Huang et al., 2022; Powers et al., 2024). This expansion is supported by data-sharing infrastructure such as MetaboLights (Haug et al., 2020) and the Human Metabolome Database (Wishart et al., 2022), which lowers barriers to data reuse and inter-laboratory comparison.

The analytical sequence involves multiple dependent stages. Phase and baseline correction affect spectral line shapes and background; normalisation adjusts for sample-to-sample dilution differences; peak alignment corrects chemical-shift drift; unsupervised methods such as Principal Component Analysis (PCA) reveal dominant variance structure and batch effects; and supervised methods such as Orthogonal Partial Least Squares Discriminant Analysis (OPLS-DA) quantify group-discriminatory variation. Cross-validation and permutation testing are needed to assess model stability and guard against overfitting, while Variable Importance in Projection (VIP) scores prioritize discriminatory resonances (Huang et al., 2022; Powers et al., 2024). Statistical Total Correlation Spectroscopy (STOCSY) identifies resonances that co-vary

with a selected anchor and thereby supports metabolite identification (Cloarec et al., 2005). A recent systematic review found substantial heterogeneity in NMR metabolomics practice, including data processing, normalisation, statistical validation, and reporting (Huang et al., 2022). When these interdependent steps are distributed across instrument software, spreadsheets, and separate statistical packages, analytical decisions are difficult to document and reconstruct, limiting reproducibility and transparent reporting (Powers et al., 2024).

As metabolomics cohort sizes grow, driven by biobank-scale studies and multi-omics integration efforts computational scalability has become an additional constraint. Datasets with tens of thousands of samples and hundreds of thousands of spectral variables can impose prohibitive memory requirements on analytical pipelines that allocate intermediate copies of the full data matrix. For example, computing Pearson correlations across all spectral variables in the STOCSY kernel by centering the entire sample-by-variable matrix requires an intermediate allocation equal in size to the input data. At 10,000 samples and 1,000,000 variables in double precision, this single intermediate array exceeds 80 gigabytes. Similarly, the VIP calculation in earlier metbit versions iterated over features in a Python loop, an O(p) sequential operation that becomes impractically slow at large feature counts. Addressing these scalability constraints without requiring users to change their analytical code motivates the compute-backend redesign in version 9.0.0.

Several open-source tools address parts of the NMR metabolomics workflow. NMRglue provides access to and manipulation of NMR data formats (Helmus and Jaroniec, 2013). pybaselines implements baseline-correction algorithms (Erb, 2022). MetaboAnalyst provides metabolomics statistics, pathway analysis, and visualisation through web and R-based interfaces (Pang et al., 2022), while NMRProcFlow supports interactive processing of one-dimensional NMR spectra (Jacob et al., 2017). These tools differ in scope, deployment model, and programming interface. metbit was developed to provide an additional Python-native option that connects commonly used NMR preprocessing, chemometric, validation, and interpretation steps within one local workflow, and version 9.0.0 extends this with explicit support for large-cohort datasets.

The present manuscript describes the design and implementation of metbit version 9.0.0, documents its compute-backend architecture and reproducible performance benchmarks, demonstrates the matrix-based workflow on a synthetic two-group dataset, and illustrates raw-data processing on the public MTBLS1 dataset. The MTBLS1 analysis is presented as a technical case study rather than an independent clinical validation because its design contains batch and repeated-measure confounding. Our aim is to establish the package's available functionality, reproducible interfaces, performance characteristics, and current limitations without extending conclusions beyond the supporting analyses.

## 2. Methods

### 2.1 Software Design

metbit (version 9.0.0) is implemented in Python 3.10 and later. It is organised as a flat-namespace package with distinct modules corresponding to stages of the NMR metabolomics workflow: nmr_preprocessing, normalisation, alignment, pca, opls_da, stocsy, and utils, together with a new large_scale module and an auto-dispatch compute backend. Module-level classes accept pandas DataFrames or NumPy arrays as primary inputs, enabling direct compatibility with standard Python data-science workflows. Each modeling class returns fitted objects that expose plots, statistics, and intermediate arrays, so downstream analyses can be driven by the same in-memory objects that generated visualisations. The package is released under the MIT license, versioned through Git and published on PyPI, and its documentation is built and hosted at https://metbit-docs.vercel.app.

### 2.2 NMR Preprocessing

Raw Bruker FID directories are handled by the nmr_preprocessing module. The pipeline removes digital filter artifacts, applies zero filling to the next power of two, executes the Fourier transform, performs automated phase correction using entropy minimisation (Chen et al., 2002), applies asymmetrically reweighted penalised least squares (arPLS) baseline correction (Baek et al., 2015) as implemented in pybaselines (Erb, 2022), and calibrates the chemical-shift axis to an internal reference. Users who have already processed their spectra in instrument software can begin directly at the normalisation or modeling stage by providing a sample-by-variable pandas DataFrame. This flexibility allows metbit to be incorporated into existing laboratory workflows without repeating upstream steps.

### 2.3 Normalisation and Alignment

metbit implements three normalisation methods. Probabilistic Quotient Normalisation (PQN; Dieterle et al., 2006) derives a sample-specific dilution factor relative to a reference spectrum, providing robust correction for unequal sample concentrations without amplifying noise. Multiplicative Scatter Correction (MSC; Martens and Stark, 1991) aligns each spectrum to a reference using a linear least-squares fit, suppressing multiplicative scatter contributions. Total Spectral Area (TSA) normalisation scales each spectrum to a fixed integral. The choice of normalisation strategy affects downstream multivariate models, and the appropriateness of each method depends on the biological matrix and expected variation structure (van den Berg et al., 2006). For chemical-shift alignment, metbit applies the interval correlation-optimised shifting algorithm (icoshift; Savorani et al., 2010), which maximizes inter-sample correlations within user-defined spectral intervals. A scikit-learn-compatible PeakAligner transformer class wraps icoshift, enabling alignment to be embedded in cross-validated pipelines or preprocessing grids. In version 9.0.0, the icoshift_align function was revised to allocate a single output array rather than creating two successive full-matrix copies, eliminating an intermediate allocation equal in size to the input matrix.

### 2.4 Principal Component Analysis

PCA is implemented via the pca class, which wraps the scikit-learn PCA decomposition with NMR-specific visualisation. Users specify the spectral matrix and a group label Series; the class fits the decomposition, computes cumulative explained variance, and generates scores plots, loading plots, and trajectory plots through Plotly. Interactive hover labels display sample identifiers, group annotations, and chemical-shift positions, supporting rapid identification of outliers and batch effects before supervised modeling.

### 2.5 OPLS-DA and Model Validation

OPLS-DA is implemented following the formulations of Trygg and Wold (2002) and Wold et al. (2001). The opls_da class separates predictive variation associated with a binary class label from orthogonal variation and returns predictive and orthogonal scores for inspection. Its internal stratified k-fold procedure reports $R^2X$, $R^2Y$, and $Q^2$. Version 9.0.0

introduces a dtype parameter that accepts numpy.float32 to halve peak memory allocation; when dtype=None (the default), float32 is selected automatically when the product of samples and features exceeds 5,000,000 elements. The class initialisation was also revised to perform NaN replacement in-place on a single numpy array allocation rather than creating a second full-matrix copy during construction. The current permutation_test utility calls scikit-learn's permutation_test_score on a PLS regression estimator. It therefore evaluates the estimator's default score and should not be interpreted as a permutation test of the custom OPLS $Q^2$. The utility is retained for exploratory use, but no permutation-derived significance claim is made in this manuscript. A future release should evaluate the complete OPLS pipeline with identical folds, scoring, grouping, and preprocessing inside each permutation. VIP scores are calculated from the fitted PLS model and are treated as variable-ranking measures rather than evidence of metabolite identity; their computation is described in Section 2.9.

## 2.6 STOCSY

STOCSY (Cloarec et al., 2005) correlates the intensity of every spectral variable against an anchor resonance selected by the user, typically a VIP-prioritised peak, across all samples. The Pearson correlation coefficient between the anchor variable and each column of the spectral matrix is computed, and a connectivity spectrum is rendered as a line plot colored by correlation magnitude. Regions of high correlation identify additional resonances belonging to the same metabolite or shared biochemical pathway, an approach that has been widely used to deconvolute metabolite spin systems from complex biological NMR spectra (Blaise et al., 2009). A local Dash application exposes STOCSY interactively in a browser: users click anchor peaks, examine connectivity, and export selected resonance lists, while the underlying computation remains in the same Python session. Identified candidate resonances can be cross-referenced with the Human Metabolome Database (Wishart et al., 2022) and the BioMagResBank (Ulrich et al., 2008) for structural assignment. For large feature counts, the ChunkedSTOCSY class described in Section 2.9 provides a memory-bounded alternative to the standard single-pass kernel.

## 2.7 Interactive Visualisation and Dash Applications

All visualisations are produced with Plotly, delivering HTML-embedded interactive outputs that can be shared without a running Python server. Four local Dash applications are included: a STOCSY explorer, a spectral annotation interface for chemical-shift regions of interest, a peak-selection tool, and a model-comparison dashboard. These applications are launched with a single function call and run on localhost, making them accessible in environments where server deployment is not feasible.

## 2.8 Large-Scale Compute Backend

Version 9.0.0 introduces a tiered auto-dispatch compute backend to address the memory and throughput constraints that arise with large metabolomics cohorts. The backend is implemented across two components: a compiled C extension (_native_backend.c) and a Python dispatch layer (_native.py). When the package is installed, the C extension is compiled with optimisation flags (-O3, -march=native, and -ffast-math). OpenMP parallelism is enabled when supported by the compiler. The build system detects OpenMP availability at installation time and gracefully falls back to a single-threaded build when OpenMP is unavailable.

The dispatch layer selects among four backends in order of priority based on the number of elements in the data matrix (n_samples × n_features) and available hardware: GPU acceleration via CuPy or PyTorch CUDA; the C extension with OpenMP parallelism for datasets exceeding 10,000,000 elements, or the single-threaded C extension below that threshold; a multiprocessing pool applied to feature chunks when the C extension is absent but multiple CPU cores are available; and a single-process chunked NumPy fallback as the guaranteed minimum. Backend selection is transparent to the calling code and can be inspected via metbit.backend_info(). Environment variables METBIT_DISABLE_NATIVE, METBIT_DISABLE_GPU, METBIT_N_JOBS, and METBIT_CHUNK allow users to override dispatch decisions without modifying scripts.

The C extension provides six kernels. pearson_columns computes Pearson correlation between an anchor column and all other columns of a matrix in a single-threaded two-pass algorithm that requires only O(p) working memory rather than the O(n × p) intermediate centred-matrix copy required by the naive NumPy implementation. pearson_columns_par parallelises the computation across rows using OpenMP, with thread-local partial-sum arrays that are reduced at the end of each parallel region. pearson_columns_f32 accepts float32 input and accumulates in float64, halving memory bandwidth for float32 datasets. column_variances and column_variances_f32 compute per-column sample variance in a numerically stable two-pass algorithm with optional OpenMP row parallelism; these functions support the feature_preselection routine described below. vip_scores computes Variable Importance in Projection scores from PLS score, weight, and loading matrices using the closed-form expression.

$$VIP_i = \sqrt{\frac{p \cdot \sum_h \left( S_h \cdot \left( \frac{w_{ih}}{\| w_{\cdot h} \|} \right)^2 \right)}{\sum_h S_h}}, \text{ where } S_h = \| \, t\_h \, \|^2 \cdot q_h^2 \quad (1)$$

This replaces the previous O(p) Python loop over features with a single vectorised kernel that is parallelised over features via OpenMP when available.

The large_scale module provides three user-facing utilities. MemoryEstimator.estimate() reports expected peak gigabytes for a dataset before any allocation is made and recommends the appropriate storage dtype. feature_preselection() removes low-variance or low-IQR spectral bins using a data-driven percentile threshold computed through the C variance kernel; this reduces dimensionality before OPLS-DA without hard-coding a threshold that would be dataset-specific. ChunkedSTOCSY computes STOCSY correlations in feature chunks, bounding peak memory to $O(n \times chunk_{size})$ regardless of total feature count while producing bit-identical results to the standard single-pass kernel.

## 2.9 VIP Score Vectorisation

Prior to version 9.0.0, VIP scores were computed by iterating over each of the p features in a Python loop, constructing a component-weight vector and evaluating the scalar VIP formula at each iteration. For p = 5,000 features, this loop required approximately 54 milliseconds of wall-clock time on the reference system. Version 9.0.0 replaces this loop with the equivalent closed-form matrix expression evaluated in a single BLAS call (numpy vectorised path) or the C kernel described above. Numerical equivalence between the loop and the vectorised result was verified at a maximum absolute difference of $4 \times 10^{-16}$, confirming that the vectorisa-

tion introduces no loss of precision relative to double-precision floating-point arithmetic.

### 2.10 Test Suite

A test suite of 305 tests accompanies version 9.0.0. The suite is organised into four complementary collections. End-to-end pipeline tests (n = 44) exercise the complete sequence from data loading through OPLS-DA fitting, VIP computation, and plot generation, verifying output types, shapes, and value ranges for each stage. Statistical validity tests (n = 22) apply the AB/AA paradigm: the AB collection verifies that OPLS-DA produces positive $Q^2$ and appropriately elevated VIP scores on a two-group synthetic dataset with a three-standard-deviation mean shift on 15 of 60 features, while the AA collection verifies that the same model does not discriminate when applied to a single population randomly split into two artificial groups. Backend dispatch tests (n = 23) confirm numerical equivalence of the C extension, multiprocessing, and NumPy fallback paths at absolute tolerance $10^{-12}$, and include memory-efficiency assertions using Python's tracemalloc module that verify peak allocation remains below 10% of the input matrix size for C-path kernels. Performance regression tests (n = 24) are marked @pytest.mark.perf and assert speedup ratios rather than absolute times, making them stable across hardware differences. All 305 tests pass on Python 3.10 through 3.13 on Linux and macOS.

### 2.11 Reproducible Benchmarking

Reproducible timing measurements for version 9.0.0 were collected using scripts/perf_report.py, a standalone benchmarking script that records the minimum of five wall-clock repetitions (each measured with time.perf_counter) following one warm-up call per configuration. All benchmarks were run on an Apple M5 Pro workstation (arm64 architecture, 15 logical CPU cores, 24 GB unified RAM) under macOS Tahoe 26.5.1 (Darwin 25.5.0), Python 3.14.5, NumPy 2.4.6, SciPy 1.17.1, scikit-learn 1.8.0, and pandas 2.3.3, with the metbit native C extension active. OpenMP parallelism was not available in this configuration because the standard macOS toolchain does not include libgomp; users who install the Homebrew libomp package before building from source will obtain an OpenMP-enabled extension. GPU acceleration was similarly not exercised because neither CuPy nor a PyTorch CUDA installation was present. Results include both timing and peak memory allocation, the latter measured via Python's tracemalloc module for one representative call per configuration. The exact environment specification is committed to the repository alongside the benchmark script, satisfying the reproducibility criteria identified in the Discussion.

### 2.12 Synthetic Workflow Example

To demonstrate the matrix-based workflow in version 9.0.0, we used a two-group synthetic NMR dataset distributed with metbit. The dataset contains 120 samples, four time points, and 65,536 variables spanning 0.0–9.0 ppm. Group differences were introduced into selected spectral regions to produce a controllable two-class structure. Workflow timing for this specific dataset was not independently re-measured for this manuscript; performance characteristics at representative dimensions are instead reported from the reproducible benchmarks described in Section 2.11. All benchmarks were executed on an Apple M5 Pro workstation (arm64 architecture, 15 logical CPU cores, 24 GB unified RAM) running macOS Tahoe 26.5.1 (Darwin kernel 25.5.0) under Python 3.14.5, NumPy 2.4.6, SciPy 1.17.1, scikit-learn 1.8.0, and pandas 2.3.3, with the metbit native C extension active (single-threaded; OpenMP and GPU acceleration not available on this configuration). The benchmark dimensions (80–500 samples, 1,000–100,000 features) bracket the synthetic example's sample count and span a range of feature counts relevant to typical NMR and LC-MS metabolomics datasets.

### 2.13 MTBLS1 Technical Case Study

To demonstrate operation on public raw data, metbit was applied to MTBLS1, a human urine $^1$H NMR study available through MetaboLights (Haug et al., 2020; Salek et al., 2007). The analysis included 132 successfully processed spectra: 48 labeled as diabetes mellitus and 84 as controls. The study metadata describe repeated collections from 30 patients and 12 healthy participants. Disease status was also completely associated with acquisition series: diabetes spectra used the ADG10003u prefix and were acquired in June 2004, whereas control spectra used ADG19007u and were acquired in May 2004. Disease and acquisition batch could therefore not be separated.

Raw Bruker FID archives were processed with digital-filter removal, zero filling, Fourier transformation, ACME phase correction (Chen et al., 2002), arPLS baseline correction (Baek et al., 2015), TSP calibration, and icoshift alignment. PQN normalisation was then applied to the complete spectral matrix, followed by Pareto-scaled PCA and a two-component OPLS-DA model. The exploratory implementation used sample-level seven-fold cross-validation; it did not group repeated observations by participant, and normalisation and alignment were not refitted within each training fold. These choices can leak information across folds and produce optimistic statistics. Accordingly, $R^2Y$, $Q^2$, predictive-score AUROC, and VIP rankings are reported only to document software output. They are not interpreted as estimates of disease discrimination or generalisability. The permutation output was excluded because the implemented test did not evaluate the same OPLS $Q^2$ statistic. Analyses were performed on an Apple M5 Pro workstation (arm64, 15 logical CPU cores, 24 GB unified RAM) running macOS Tahoe 26.5.1 (Darwin 25.5.0) under Python 3.14.5, NumPy 2.4.6, SciPy 1.17.1, scikit-learn 1.8.0, pandas 2.3.3, and metbit 9.0.0 with the native C extension active.

## 3. Results

### 3.1 Workflow Overview

The metbit workflow progresses through common tabular structures. A spectral matrix enters normalisation and alignment, PCA summarizes major variance, OPLS-DA provides exploratory supervised modeling, and VIP scoring and STOCSY support variable prioritisation and correlation-based spectral inspection. Because each stage accepts standard DataFrames or arrays, the workflow can be recorded and embedded in analysis scripts. Valid inferential use nevertheless requires study-specific control of batch effects, participant grouping, and fold-wise preprocessing.

### 3.2 Version 9.0.0 Compute-Backend Performance

| Kernel | Dataset (n × p) | Implementation | Min time (ms) | Peak RAM (MB) | Speedup | RAM reduction |
|---|---|---|---|---|---|---|
| VIP scores | 80 × 1,000 | Python loop (baseline) | 6.6 | | 1.0× | |
| | 80 × 1,000 | NumPy vectorised | 0.02 | | 322× | |
| | 80 × 1,000 | C dispatch | 0.005 | | 1,347× | |
| | 80 × 5,000 | Python loop (baseline) | 54.0 | | 1.0× | |
| | 80 × 5,000 | NumPy vectorised | 0.07 | | 817× | |
| | 80 × 5,000 | C dispatch | 0.02 | | 2,680× | |
| | 80 × 20,000 | Python loop (baseline) | (skipped) | | | |
| | 80 × 20,000 | NumPy vectorised | 0.24 | | | |
| | 80 × 20,000 | C dispatch | 0.08 | | | |
| Pearson / STOCSY | 200 × 10,000 | Full-copy NumPy (baseline) | 1.1 | 16.3 | 1.0× | |
| | 200 × 10,000 | C dispatch (float64) | 0.9 | 0.3 | 1.2× | 51× |
| | 200 × 10,000 | C dispatch (float32) | 0.7 | 0.3 | 1.5× | 51× |
| | 500 × 30,000 | Full-copy NumPy (baseline) | 8.3 | 121.0 | 1.0× | |
| | 500 × 30,000 | C dispatch (float64) | 6.6 | 1.0 | 1.2× | 126× |
| | 500 × 30,000 | C dispatch (float32) | 6.0 | 1.0 | 1.4× | 126× |
| Column variance | 200 × 50,000 | NumPy (baseline) | 6.0 | 80.8 | 1.0× | |
| | 200 × 50,000 | C dispatch (float64) | 3.5 | 0.8 | 1.7× | 101× |
| | 200 × 50,000 | C dispatch (float32) | 3.0 | 0.8 | 2.0× | 101× |
| | 500 × 100,000 | NumPy (baseline) | 30.3 | 401.6 | 1.0× | |
| | 500 × 100,000 | C dispatch (float64) | 17.6 | 1.6 | 1.7× | 251× |
| | 500 × 100,000 | C dispatch (float32) | 15.4 | 1.6 | 2.0× | 251× |
| ChunkedSTOCSY | 50 × 2,000 | Standard STOCSY | 6.8 | | 1.0× | |
| | 50 × 2,000 | ChunkedSTOCSY | 0.2 | | 34× | |
| | 100 × 5,000 | Standard STOCSY | 28.5 | | 1.0× | |
| | 100 × 5,000 | ChunkedSTOCSY | 0.6 | | 47× | |

Table 1. Reproducible performance benchmarks for version 9.0.0 compute kernels. Times are minimum of five wall-clock repetitions (one warm-up call discarded). Peak memory is measured by Python's tracemalloc module for one representative call. Speedup and memory-reduction ratios are computed relative to the baseline implementation. Apple M5 Pro, arm64, 15 logical cores, 24 GB RAM, macOS Tahoe 26.5.1, Python 3.14.5, NumPy 2.4.6, SciPy 1.17.1, metbit 9.0.0 C extension active, OpenMP and GPU not available.

The VIP score kernel shows the most pronounced speedup because the previous implementation was a Python-level loop over features that invoked NumPy operations individually at each iteration. At p = 5,000 features, the Python loop required 54 milliseconds; the C dispatch kernel completed the same computation in 0.02 milliseconds, a 2,680-fold improvement. At p = 20,000 features, the Python loop was too slow to time in a reasonable duration and was skipped; the C dispatch kernel required 0.08 milliseconds. These speedups are consistent across repetitions because the computation is dominated by the loop overhead rather than by memory access patterns.

The Pearson correlation kernel used by STOCSY shows a qualitatively different improvement profile. On the Apple M5 Pro processor, the NumPy Basic Linear Algebra Subprogram (BLAS) implementation backed by the Accelerate framework is highly optimised, limiting the speed advantage of the C extension to 1.2-1.5-fold at the tested dimensions. The dominant advantage is memory: the full-copy NumPy baseline materialises an (n × p) centred matrix as an intermediate result, whereas the C extension maintains only O(p) working arrays. At 500 samples and 30,000 features in double precision, the baseline peak allocation is 121.0 MB compared with 1.0 MB for the C extension, a 126-fold reduction. This memory advantage grows proportionally with dataset size; at 500 samples and 1,000,000 features (a realistic large-cohort NMR dataset), the baseline would require approximately 4 GB of intermediate allocation, whereas the C extension would require approximately 24 MB. The column variance kernel used by feature_preselection shows analogous characteristics, with a 251-fold peak-memory reduction at 500 samples and 100,000 features in addition to a 2.0-fold speed advantage for the float32 path.

ChunkedSTOCSY computes Pearson correlations in feature batches, bounding peak memory to O(n × chunksize). The 47-fold speed advantage over the standard STOCSY path at 100 samples and 5,000 features arises because the standard path incurs Python-level overhead from Plotly figure construction and data marshalling that is absent from the lightweight ChunkedSTOCSY.compute() method. Both implementations produce bit-identical correlation and p-value outputs, as confirmed by the test suite at an absolute tolerance of $10^{-12}$.

### 3.3 Research Adoption

metbit has been applied in two peer-reviewed clinical NMR metabolomics studies. Karunasumetta et al. (2024a) used metbit to characterize metabolomic differences between on-pump and off-pump coronary artery bypass grafting (CABG) in a prospective cohort of 21 patients; the package was cited explicitly by its GitHub repository in the statistical methods section and used alongside SIMCA 14 for OPLS-DA modeling, permutation testing, and VIP-based variable selection. A second study by the same group (Karunasumetta et al., 2024b) applied the same metbit-based pipeline to investigate metabolomic signatures associated with different cardioplegic solutions in cardiac surgery. Together, these publications constitute the first peer-reviewed application record for the pack-

age in a clinical metabolomics context. Both papers were co-authored by members of the metbit development team. An independent application was additionally identified in an undergraduate thesis submitted to the Faculdade de Ciências Farmacêuticas de Ribeirão Preto, Universidade de São Paulo (Mencucini, 2025). In that work, metbit was applied alongside standard Python data-science tools to multivariate analysis of mass spectrometry-based metabolomics feature matrices, demonstrating that the package is used beyond NMR and by researchers outside the development group.

Distribution metrics provide additional evidence of package activity. When retrieved on 19 June 2026, PyPi reported 154,587 cumulative downloads for metbit, while mirror-excluded PyPI Stats records showed 1,340 downloads during the preceding 90 days and 443 during the preceding 30 days (data available through 18 June 2026). Download figures should be interpreted cautiously because automated CI/CD systems and repeated installations contribute to the counts. These values indicate distribution activity rather than unique users, active installations, research use, or software quality.

### 3.4 Comparison with Existing Tools

| Feature | metbit | MetaboAnalyst | NMRProcFlow | NMRglue | pybaselines |
|---|---|---|---|---|---|
| NMR preprocessing | Yes | Partial | Yes | Yes | No |
| PQN / MSC normalisation | Yes | Yes | Yes | No | No |
| icoshift alignment | Yes | No | Yes | No | No |
| PCA | Yes | Yes | Yes | No | No |
| OPLS-DA | Yes | Yes | No | No | No |
| Permutation utility | Exploratory PLS score | Yes | No | No | No |
| VIP scoring | Yes | Yes | No | No | No |
| STOCSY | Yes | No | No | No | No |
| Python-native scripting | Yes | No (R interface available) | No | Yes | Yes |
| Local data processing | Yes | Web: No; R: Yes | Yes | Yes | Yes |
| Interactive Dash apps | Yes | No | No | No | No |
| Large-cohort C/GPU backend | Yes (v9.0.0) | No | No | No | No |
| Memory-bounded ChunkedSTOCSY | Yes (v9.0.0) | No | No | No | No |
| float32 storage mode | Yes (v9.0.0) | No | No | No | No |
| Feature pre-selection | Yes (v9.0.0) | Yes | No | No | No |

Table 2. Descriptive feature comparison based on cited publications and project materials reviewed in June 2026. The table is not an independent benchmark; current releases and extensions may differ. Version 9.0.0 additions are noted.

### 3.5 MTBLS1 Case-Study Output

metbit processed all 132 selected Bruker FID archives and produced a matrix of 50,029 chemical-shift variables. Mean absolute TSP calibration deviation was 4.12 ± 2.18 m-ppm, and the study-specific signal-to-noise calculation yielded a mean of 244.2. Raw-data preprocessing required 39.1 seconds, PCA required 0.23 seconds, and OPLS-DA fitting required 50.9 seconds; these are single-run case-study timings recorded on the Apple M5 Pro system specified in Section 2.13 (macOS Tahoe 26.5.1, Python 3.14.5, NumPy 2.4.6, metbit 9.0.0).

The exploratory sample-level OPLS-DA output was $R^2Y = 0.998$, $Q^2 = 0.340$, and predictive-score AUROC = 0.931. These values are not estimates of clinical performance. Disease status was completely confounded with acquisition series, repeated observations from the same participants were allowed across folds, and preprocessing was performed before cross-validation. The combination of near-perfect apparent fit and substantially lower $Q^2$ is consistent with an optimistic or unstable model and does not establish disease-specific discrimination. The previously calculated permutation result was removed because it did not test the same OPLS $Q^2$ statistic.

High VIP values occurred in regions compatible with resonances reported for glucose, citrate, taurine, and creatinine. These labels are tentative spectral annotations only. The present analysis did not confirm them through STOCSY connectivity, reference-spectrum matching, multiplicity analysis, spiking, or two-dimensional NMR. Moreover, acquisition batch, sex, age, BMI, and repeated sampling may contribute to the ranked variables. The case study therefore demonstrates that metbit can process the archived data and generate exploratory outputs, but it does not validate biomarkers or a diabetes classifier.

## 4. Discussion

metbit addresses a practical reproducibility challenge in NMR metabolomics: the fragmentation of analytical steps across multiple software environments. By consolidating preprocessing, normalisation, alignment, PCA, exploratory OPLS-DA, VIP scoring, STOCSY, and interactive visualisation within a Python package, metbit allows an analytical sequence to be recorded, version-controlled, and rerun from a script. Version 9.0.0 extends this foundation with a scalable compute backend that addresses memory and throughput constraints arising in large metabolomics cohorts. Software integration does not by itself ensure valid inference, however. Recent reviews emphasize that reproducible metabolic phenotyping also requires explicit control of study design, batch effects, repeated observations, preprocessing leakage, model selection, and validation procedures (Blaise et al., 2021; Huang et al., 2022; Powers et al., 2024).

The version 9.0.0 performance benchmarks demonstrate two complementary improvements. The VIP score kernel replaces a Python loop over features with a closed-form matrix expression evaluated by the C extension, yielding 2,680-fold and 817-fold speedups relative to the loop at p = 5,000 features for the C-dispatch and NumPy vectorised paths respectively. These results are not hardware-dependent in the sense that any loop-dominated computation incurs the same Python interpreter overhead regardless of processor architecture. The Pearson correlation kernel used by STOCSY, in contrast, shows a more modest speed advantage of 1.2-1.5-fold on the Apple M5 Pro because the Accelerate-backed NumPy BLAS is already highly optimised on that platform. The primary and practically important benefit of the C implementation is memory: the baseline full-copy approach allocates an (n × p) intermediate centred matrix, whereas the C kernel operates with O(p) working arrays. At 500 samples and 30,000 features, this corresponds to a 126-fold reduction in peak allocation from 121 MB to 1 MB. The memory benefit scales proportionally with dataset size, making the C backend essential rather than merely preferable for cohorts with tens of thousands of samples or hundreds of thousands of features. For users without a compiled C extension for example, in restricted computing environments, the multiprocessing and chunked NumPy fallbacks provide the same O(p)-per-chunk memory profile at a throughput penalty, ensuring that analyses complete without out-of-memory errors on machines with limited RAM.

The ChunkedSTOCSY class addresses a related constraint. At large feature counts, the standard STOCSY implementation constructs a full Plotly figure object with one data point per feature, which itself becomes a bottleneck in Python object allocation. ChunkedSTOCSY separates correlation computation from visualisation and processes features in configurable batches, producing bit-identical results to the standard kernel while bounding peak memory and providing a 47-fold throughput advantage at 100 samples and 5,000 features. Users working with datasets at scales where the standard STOCSY completes in under a second will notice no meaningful difference; the ChunkedSTOCSY path becomes important when feature counts exceed approximately 100,000 on a typical workstation.

The reproducible benchmark methodology introduced in version 9.0.0 represents a deliberate improvement over the single-run timing reported in the original manuscript, which did not record repeated-run dispersion, warm-up conditions, or the complete software environment. The current benchmarks use five repetitions with one discarded warm-up, report minimum times alongside peak memory, archive results to a JSON file for trend comparison, and commit the benchmark script and environment specification to the repository. This approach is consistent with the criteria for performance reproducibility identified in the Discussion of this manuscript and in recent guidance on scientific computing benchmarking. Users wishing to reproduce or extend the benchmarks can run python scripts/perf_report.py from the repository root after installation. The reported values should nonetheless be interpreted as characterising one specific hardware and software environment; performance on other systems will differ, particularly for the OpenMP and GPU paths that were not exercised in the current benchmarks.

The test suite of 305 tests addresses a gap in earlier releases, which lacked systematic verification of statistical validity, dispatch routing, and numerical equivalence across backends. The AB/AA statistical validity paradigm, verifying that the model discriminates genuine signal and fails to discriminate noise, provides a data-driven correctness criterion complementary to unit tests of individual functions. The memory-efficiency assertions using tracemalloc verify the primary design claim of the C kernels: that peak allocation scales with O(p) working arrays rather than O(n × p) input data. These tests are designed to catch regressions that would silently restore the memory-expensive code path without breaking numerical outputs.

A key design decision in version 9.0.0 is acceptance of a sample-by-variable spectral matrix as the primary input with automatic backend selection. This vendor-neutral entry point allows researchers to use the chemometric and interpretation functions after laboratory-specific upstream processing. The optional raw-data module supports Bruker FID directories but is not required. This flexibility makes metbit complementary to NMRProcFlow, MetaboAnalyst, NMRglue, pybaselines, and other tools that address different workflow stages (Table 2). The comparison is descriptive and does not establish that one package is superior; a defensible comparative evaluation would require predefined tasks, current versions, common datasets, reproducibility criteria, and independent assessors.

PLS and OPLS are established chemometric approaches, but their validity depends on implementation, model complexity, preprocessing, and study design rather than the method name alone (Blaise et al., 2021; Debik et al., 2022). VIP scores can help rank variables, but they do not provide confidence intervals, multiplicity control, or structural identification (Debik et al., 2022; Powers et al., 2024). STOCSY can add evidence by identifying correlated resonances, yet correlation may reflect shared dilution, batch effects, biological covariance, or overlapping signals. Candidate identities should therefore be confirmed with reference spectra, resonance multiplicity, spiking experiments, or two-dimensional NMR where feasible (Moco, 2022; Gowda and Raftery, 2023; Powers et al., 2024).

The study has several important limitations. First, the current OPLS-DA implementation is restricted to binary classification. Second, its permutation utility does not test the custom OPLS $Q^2$ statistic and should not support inferential claims in its present form; a future release should permute labels and refit the complete OPLS pipeline using the same grouping, folds, preprocessing, component-selection procedure, and $Q^2$ definition as the observed model, consistent with recent statistical guidance (Blaise et al., 2021; Debik et al., 2022; Powers et al., 2024). Third, the MTBLS1 case study cannot separate disease status from acquisition batch: all diabetes and control spectra belong to different acquisition series and periods. The dataset also contains repeated observations, but cross-validation was performed at the spectrum level rather than the participant level. In addition, alignment and PQN normalisation were applied before cross-validation. These factors can substantially inflate apparent model performance, as highlighted in recent guidance on metabolomics study design and multivariate analysis (Blaise et al., 2021;

Huang et al., 2022; Debik et al., 2022; Powers et al., 2024). A valid predictive analysis would require participant-grouped validation, fold-specific preprocessing, nested component selection, and ideally an independent cohort in which disease and acquisition batch are not confounded. Fourth, the performance benchmarks did not exercise OpenMP or GPU paths because these resources were not available on the benchmark system; users with OpenMP-enabled builds or CUDA-capable GPUs may observe substantially different and potentially larger improvements than those reported in Table 1.

Additional limitations concern interpretation and adoption. The reported VIP regions in the MTBLS1 case study were not confirmed by STOCSY, reference matching, spiking, or two-dimensional NMR and are therefore tentative annotations. The feature comparison in Table 2 is descriptive rather than independently benchmarked. Two peer-reviewed studies using metbit were co-authored by members of the development team, while one undergraduate thesis represents a documented independent application (Karunasumetta et al., 2024a, 2024b; Mencucini, 2025). PyPI download counts may include automated systems and repeated installations and should be treated as distribution activity rather than evidence of scientific adoption.

The local Plotly and Dash interfaces permit exploratory inspection without uploading potentially sensitive data to an external web service. Because the interfaces operate on the same Python objects used by the scripted analysis, users can preserve the computational context of interactive exploration. Reproducing the same output still depends on versioned code, data, parameters, random seeds, and environment specifications, consistent with current best-practice recommendations for NMR metabolomics (Huang et al., 2022; Powers et al., 2024). The MIT license permits groups to inspect and adapt the implementation for their NMR platforms, biological matrices, and governance requirements.

## Research Impact Statement

metbit provides an openly available, Python-native environment for connecting common NMR metabolomics operations. Its principal impact is infrastructural: analyses can be scripted, inspected, versioned, and executed locally from raw Bruker data or standardised spectral matrices. Version 9.0.0 additionally provides a scalable compute backend that reduces peak memory allocation by 50-251-fold for key kernels relative to naive NumPy implementations, making the package tractable for large-cohort studies without requiring hardware changes or code modifications. The MTBLS1 case study demonstrates technical operation on public human NMR data but does not establish clinical validity because disease status is confounded with acquisition batch and the exploratory validation design permits leakage. Published applications and an independent academic thesis provide early evidence of use. When retrieved on 19 June 2026, PyPi reported 154,587 cumulative downloads, while mirror-excluded PyPI Stats records showed 1,340 downloads over the preceding 90 days and 443 over the preceding 30 days. These figures reflect distribution activity rather than unique users or scientific adoption. A test suite of 305 tests, including AB/AA statistical validity checks and tracemalloc-based memory assertions, accompanies version 9.0.0. The MIT license, PyPI distribution, and documentation support method inspection, teaching, and extension. Future impact depends on correcting the permutation procedure, enabling participant-grouped and leakage-resistant validation, expanding independent testing, benchmarking on OpenMP and GPU configurations, and accumulating independent publications from groups outside the development team.

## Conclusion

metbit version 9.0.0 integrates NMR-oriented preprocessing, normalisation, alignment, PCA, exploratory OPLS-DA, VIP ranking, STOCSY, and interactive visualisation within a Python-native package that operates on standard tabular structures. The release introduces a four-tier auto-dispatch compute backend—GPU, C+OpenMP, multiprocessing, and chunked NumPy—that reduces peak memory allocation by 50-251-fold for the Pearson correlation and column variance kernels and accelerates VIP score computation by up to 2,680-fold relative to the previous Python-loop implementation, without requiring any change to calling code. A ChunkedSTOCSY class bounds STOCSY memory to a user-configurable chunk size, and a feature_preselection function provides dimensionality reduction through a C-accelerated variance kernel before supervised modeling. A test suite of 305 tests verifies correctness across backends using numerical equivalence assertions, AB/AA statistical validity checks, and tracemalloc-based memory-efficiency assertions. The synthetic example and MTBLS1 case study demonstrate workflow execution, not validated predictive performance. The current evidence supports the package as an open and extensible analysis environment for datasets spanning from small clinical cohorts to large multi-thousand-sample studies, while clinical discrimination, biomarker identification, comparative superiority, and performance under OpenMP or GPU acceleration remain incompletely characterised. Addressing grouped validation, fold-specific preprocessing, permutation scoring, metabolite confirmation, and expanded hardware benchmarking will be necessary for stronger claims. metbit is available under the MIT license at https://github.com/aeiwz/metbit, distributed through PyPI, and documented at https://metbit-docs.vercel.app.

### Author Contributions

Theerayut Bubpamala: Conceptualisation; Software; Formal Analysis; Writing - Original Draft; Writing - Review and Editing; Visualisation; Project Administration.
Chotika Chatgasem: Software; Contributing Development.


## Acknowledgements

The authors gratefully acknowledge the open-source scientific Python community for the foundational libraries on which metbit is built, including NumPy, SciPy, Pandas, scikit-learn, Plotly, and Dash. The authors thank the EMBL-EBI MetaboLights team and the original investigators of study MTBLS1 for making publicly accessible metabolomics datasets available for independent validation. C.C. is acknowledged for contributions to software design and testing.

### Conflict of Interest

The authors declare no conflicts of interest.

### Funding

metbit is independently developed and maintained. This work received no external funding. The article processing charge, if applicable, will be supported by the authors.


### Ethics Statement

This manuscript makes use of MTBLS1, a publicly archived de-identified human urine NMR metabolomics dataset available through MetaboLights (study identifier MTBLS1). The original study was conducted with ethics approval and informed consent

as described in Salek et al. (2007). No new human participants were recruited for this work; all analyses used only the publicly available, de-identified FID archives and associated metadata as provided by MetaboLights under its open-access terms. Secondary analysis of publicly archived de-identified data of this type does not require independent ethics review under the guidelines applicable to the authors' institutions.

**Data Availability**

metbit is freely available under the MIT License at https://github.com/aeiwz/metbit. Versioned releases are distributed through https://pypi.org/project/metbit/ and can be installed with pip install metbit. Documentation, API reference, and worked examples are available at https://metbit-docs.vercel.app. The benchmark dataset used in this study is included in the project repository. The performance benchmark script, result archive (reports/benchmark_results.json), and environment specification used to generate the results in Table 1 are available in the scripts/ and reports/ directories of the version 9.0.0 release tag. The exact scripts, random seeds, and environment specification used to generate the MTBLS1 results are available in the manuscript/benchmark/ directory of the repository.

## References


Baek SJ, Park A, Ahn YJ, Choo J. Baseline correction using asymmetrically reweighted penalised least squares smoothing. Analyst 2015;140:250-7. https://doi.org/10.1039/C4AN01061B

Beckonert O, Keun HC, Ebbels TMD, Bundy J, Holmes E, Lindon JC, Nicholson JK. Metabolic profiling, metabolomic and metabonomic procedures for NMR spectroscopy of urine, plasma, serum and tissue extracts. Nat Protoc 2007;2:2692-703. https://doi.org/10.1038/nprot.2007.376

van den Berg RA, Hoefsloot HCJ, Westerhuis JA, Smilde AK, van der Werf MJ. Centering, scaling, and transformations: improving the biological information content of metabolomics data. BMC Genomics 2006;7:142. https://doi.org/10.1186/1471-2164-7-142

Blaise BJ, Gaubert G, Thabuis C, Gilles C, Haas C, Guillot D, Favre P, Lacassagne MN, Cren-Olive C, Emsley L, Toulhoat H. Statistical recoupling prior to significance testing in NMR-based metabolomics. Anal Chem 2009;81:6242-51. https://doi.org/10.1021/ac900509e

Blaise BJ, Correia GDS, Haggart GA, Surowiec I, Sands C, Lewis MR, Pearce JTM, Trygg J, Nicholson JK, Holmes E, Ebbels TMD. Statistical analysis in metabolomics. Annu Rev Anal Chem 2021;14:271-99. https://doi.org/10.1146/annurev-anchem-091620-091959

Broadhurst D, Goodacre R, Reinke SN, Kuligowski J, Wilson ID, Lewis MR, Dunn WB. Guidelines and considerations for the use of system suitability and quality control samples in mass spectrometry assays applied in untargeted clinical metabolomic studies. Metabolomics 2018;14:72. https://doi.org/10.1007/s11306-018-1367-3

Chen L, Weng Z, Goh L, Garland M. An efficient algorithm for automatic phase correction of NMR spectra based on entropy minimisation. J Magn Reson 2002;158:164-8. https://doi.org/10.1016/S1090-7807(02)00069-1

Cloarec O, Dumas ME, Craig A, Barton RH, Trygg J, Hudson J, Blancher C, Gauguier D, Lindon JC, Holmes E, Nicholson JK. Statistical total correlation spectroscopy: an exploratory approach for latent biomarker identification from metabolic $^{1}$H NMR data sets. Anal Chem 2005;77:1282-9. https://doi.org/10.1021/ac048630x

Debik J, Sangermani M, Wang F, Madssen TS, Giskeodegard GF. Multivariate analysis of NMR-based metabolomic data. NMR Biomed 2022;35:e4638. https://doi.org/10.1002/nbm.4638

Dieterle F, Ross A, Schlotterbeck G, Senn H. Probabilistic quotient normalisation as a robust method to account for dilution of complex biological mixtures. Application in $^{1}$H NMR metabonomics. Anal Chem 2006;78:4281-90. https://doi.org/10.1021/ac051632c

Emwas AH, Roy R, McKay RT, Tenori L, Saccenti E, Gowda GAN, Raftery D, Alahmari F, Jaremko L, Jaremko M, Wishart DS. NMR spectroscopy for metabolomics research. Metabolites 2019;9:123. https://doi.org/10.3390/metabo9070123

Erb A. pybaselines: A Python library of algorithms for the baseline correction of experimental data. J Open Source Softw 2022;7:4554. https://doi.org/10.21105/joss.04554

Eriksson L, Trygg J, Wold S. CV-ANOVA for significance testing of PLS and OPLS models. J Chemom 2008;22:594-600. https://doi.org/10.1002/cem.1187

Fiehn O, Kristal B, van Ommen B, Sumner LW, Sansone SA, Taylor C, Hardy N, Kaddurah-Daouk R. Establishing reporting standards for metabolomic and metabonomic studies: a call for participation. OMICS 2007;10:158-63. https://doi.org/10.1089/omi.2006.0008

Gowda GAN, Raftery D. NMR metabolomics methods for investigating disease. Anal Chem 2023;95:83-99. https://doi.org/10.1021/acs.analchem.2c04606

Gromski PS, Muhamadali H, Ellis DI, Xu Y, Correa E, Turner ML, Goodacre R. A tutorial review: Metabolomics and partial least squares-discriminant analysis - a marriage of convenience or a shotgun wedding. Anal Chim Acta 2015;879:10-23. https://doi.org/10.1016/j.aca.2015.02.012

Harris CR, Millman KJ, van der Walt SJ et al. Array programming with NumPy. Nature 2020;585:357-62. https://doi.org/10.1038/s41586-020-2649-2

Haug K, Cochrane K, Nainala VC, Williams M, Chang J, Jayaseelan KV, O'Donovan C. MetaboLights: a resource evolving in response to the needs of its scientific community. Nucleic Acids Res 2020;48:D440-4. https://doi.org/10.1093/nar/gkz1019

Helmus JJ, Jaroniec CP. Nmrglue: an open source Python package for the analysis of multidimensional NMR data. J Biomol NMR 2013;55:355-67. https://doi.org/10.1007/s10858-013-9718-x

Huang K, Thomas N, Gooley PR, Armstrong CW. Systematic review of NMR-based metabolomics practices in human disease research. Metabolites 2022;12:963. https://doi.org/10.3390/metabo12100963

Jacob D, Deborde C, Lefebvre M, Maucourt M, Moing A. NMRProcFlow: a graphical and interactive tool dedicated to 1D spectra processing for NMR-based metabolomics. Metabolomics 2017;13:36. https://doi.org/10.1007/s11306-017-1144-5

Karunasumetta C, Tourthong W, Mala R, Chatgasem C, Bubpamala T, Punchai S, Sawanyawisuth K. Comparative analysis of metabolomic responses in on-pump and off-pump coronary artery bypass graft surgery. Metabolomics 2024;20. https://doi.org/10.1007/s11306-024-02181-6

Karunasumetta C, Chatgasem C, Punchai S, Sawanyawisuth K. Metabolomic signatures influenced by different cardioplegic solutions in cardiac surgery. J Pract Cardiovasc Sci 2024;10:181-8. https://doi.org/10.4103/jpcs.jpcs_63_24

Lindon JC, Nicholson JK. Spectroscopic and statistical techniques for information recovery in metabonomics and metabolomics. Annu Rev Anal Chem 2008;1:45-69. https://doi.org/10.1146/annurev.anchem.1.031207.113026

Markley JL, Brüschweiler R, Edison AS, Eghbalnia HR, Powers R, Raftery D, Wishart DS. The future of NMR-based metabolomics. Curr Opin Biotechnol 2017;43:34-40. https://doi.org/10.1016/j.copbio.2016.08.001

Martens H, Stark E. Extended multiplicative signal correction and spectral interference subtraction: new preprocessing methods for near infrared spectroscopy. J Pharm Biomed Anal 1991;9:625-35. https://doi.org/10.1016/0731-7085(91)80188-F

Mencucini LGS. Ferramentas em Python para pre-processamento e analise de dados de metabolomica baseada em espectrometria de massas [Undergraduate thesis]. Sao Paulo: Faculdade de Ciencias Farmaceuticas, Universidade de Sao Paulo; 2025.

Moco S. Studying metabolism by NMR-based metabolomics. Front Mol Biosci 2022;9:882487. https://doi.org/10.3389/fmolb.2022.882487

Nicholson JK, Lindon JC, Holmes E. Metabonomics: understanding the metabolic responses of living systems to pathophysiological stimuli via multivariate statistical analysis of biological NMR spectroscopic data. Xenobiotica 1999;29:1181-9. https://doi.org/10.1080/004982599238047

Pang Z, Chong J, Zhou G, de Lima Morais DA, Chang L, Barrette M, Gauthier C, Jacques PE, Li S, Xia J. MetaboAnalyst 5.0: narrowing the gap between raw spectra and functional insights. Nucleic Acids Res 2022;50:W590-8. https://doi.org/10.1093/nar/gkac317

Pedregosa F, Varoquaux G, Gramfort A et al. Scikit-learn: Machine learning in Python. J Mach Learn Res 2011;12:2825-30.

Powers R, Andersson ER, Bayless AL, Brua RB, Chang MC, Cheng LL et al. Best practices in NMR metabolomics: Current state. TrAC Trends Anal Chem 2024;171:117473. https://doi.org/10.1016/j.trac.2023.117473

Saccenti E, Hoefsloot HCJ, Smilde AK, Westerhuis JA, Hendriks MMWB. Reflections on univariate and multivariate analysis of metabolomics data. Metabolomics 2014;10:361-74. https://doi.org/10.1007/s11306-013-0598-6

Salek RM, Maguire ML, Bentley E, Rubtsov DV, Hough T, Cheeseman M, Nunez D, Sweatman BC, Haselden JN, Cox RD, Connor SC, Griffin JL. A metabolomic comparison of urinary changes in type 2 diabetes in mouse, rat, and human. Physiol Genomics 2007;29:99-108. https://doi.org/10.1152/physiolgenomics.00194.2006

Savorani F, Tomasi G, Engelsen SB. icoshift: A versatile tool for the rapid alignment of 1D NMR spectra. J Magn Reson 2010;202:190-202. https://doi.org/10.1016/j.jmr.2009.11.012

Sumner LW, Amberg A, Barrett D et al. Proposed minimum reporting standards for chemical analysis. Metabolomics 2007;3:211-21. https://doi.org/10.1007/s11306-007-0082-2

Triba MN, Le Moyec L, Amathieu R, Goossens C, Bouchemal N, Nahon P, Rutledge DN, Savarin P. PLS/OPLS models in metabolomics: the impact of permutation of dataset prior to cross-validation. Mol Biosyst 2015;11:13-9. https://doi.org/10.1039/C4MB00414K

Trygg J, Wold S. Orthogonal projections to latent structures (O-PLS). J Chemom 2002;16:119-28. https://doi.org/10.1002/cem.695

Ulrich EL, Akutsu H, Doreleijers JF et al. BioMagResBank. Nucleic Acids Res 2008;36:D402-8. https://doi.org/10.1093/nar/gkm957

Virtanen P, Gommers R, Oliphant TE et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nat Methods 2020;17:261-72. https://doi.org/10.1038/s41592-019-0686-2

Westerhuis JA, Hoefsloot HCJ, Smit S, Vis DJ, Smilde AK, van Velzen EJJ, van Duijnhoven JPM, van Dorsten FA. Assessment of PLSDA cross validation. Metabolomics 2008;4:81-9. https://doi.org/10.1007/s11306-007-0099-6

Wishart DS, Guo A, Oler E et al. HMDB 5.0: the Human Metabolome Database for 2022. Nucleic Acids Res 2022;50:D622-31. https://doi.org/10.1093/nar/gkab1062

Wold S, Johansson E, Cocchi M. PLS: Partial Least Squares Projections to Latent Structures. In: Kubinyi H, editor. 3D QSAR in Drug Design. Leiden: ESCOM; 1993. p. 523-50.

Wold S, Sjostrom M, Eriksson L. PLS-regression: a basic tool of chemometrics. Chemom Intell Lab Syst 2001;58:109-30. https://doi.org/10.1016/S0169-7439(01)00155-1

Worley B, Powers R. Multivariate Analysis in Metabolomics. Curr Metabolomics 2013;1:92-107. https://doi.org/10.2174/2213235X11301010092